

Restaurant Automation SaaS - API Documentation

1. Purpose

This document is the copy-paste reference for the backend API surface of the Restaurant Automation SaaS platform. It aligns with the role-wise PRD and the role-wise Postman collection so the backend, frontend, and testing flow stay consistent.

Source of truth:

- restaurant_automation_final_prd.md
- restaurant_automation_rolewise_postman_and_prd.md

The platform uses mobile-OTP customer authentication for account access and JWT-based staff/admin flow:

- Customers can authenticate with mobile number + OTP for account access, while dine-in ordering still uses temporary table sessions via `x-session-token`
 - Staff, restaurant admins, and super admin use JWT auth via `Authorization: Bearer <token>`
-

2. API Standards

Base URL

All HTTP APIs are versioned under:

`/api/v1`

Standard success response

```
{
  "success": true,
  "data": {}
}
```

Standard error response

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed"
  }
}
```

Common error codes

- VALIDATION_ERROR
- INVALID_REQUEST
- UNAUTHORIZED
- TOKEN_EXPIRED
- TOKEN_INVALID
- FORBIDDEN
- NOT_FOUND
- CONFLICT
- RATE_LIMIT_EXCEEDED
- TABLE_SESSION_EXPIRED
- INTERNAL_ERROR
- INVALID_OTP
- OTP_EXPIRED
- OTP_ATTEMPTS_EXCEEDED
- TENANT_VIOLATION
- PASSWORD_RESET_REQUIRED
- PAYMENT_VERIFICATION_FAILED

Query param conventions

- Use path params for a single resource, for example `/items/:id`
 - Use query params for filtering, sorting, pagination, and dashboards, for example:
 - `?page=1&limit=20`
 - `?status=READY`
 - `?search=burger`
 - `?from=2026-01-01&to=2026-01-31`
-

3. Role-wise Postman Collection Structure

Restaurant Automation API

- Auth APIs
- Public APIs
- Customer APIs
- Service Staff APIs
- Kitchen APIs
- Cleaning APIs
- Restaurant Admin APIs
- Super Admin APIs
- Shared Utility APIs
- Webhook APIs
- Socket Testing APIs
- Health & Monitoring APIs

4. Authentication Model

4.1 Staff / Admin / Kitchen / Cleaning

Use JWT auth:

- Authorization: Bearer <accessToken>

4.2 Customer dine-in flow

Use QR/table sessions:

- x-session-token: <sessionToken>

4.3 Customer authentication and onboarding

Customer login is mobile-OTP based.

Rules:

- Customers do not have passwords.
 - Customers do not register using email.
 - Customer login is performed only through mobile number + OTP.
 - Customer onboarding requires only the customer's name and mobile number.
 - OTP verification creates/authenticates the customer account.
-

5. Auth APIs

Method	Path	Purpose
POST	/auth/register	Create a staff user account
POST	/auth/login	Login with password or supported credentials
POST	/auth/request-otp	Request customer OTP
POST	/auth/verify-otp	Verify customer OTP
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Logout and clear session state
POST	/auth/forgot-password	Start staff password recovery
POST	/auth/verify-reset-otp	Verify staff password reset OTP
POST	/auth/reset-password	Complete staff password reset
GET	/auth/me	Get current authenticated user
GET	/auth/sessions	List active sessions
DELETE	/auth/sessions/:sessionId	Revoke one session

5.1 Customer Authentication

Request OTP POST /auth/request-otp

Request:

```
{
  "mobile": "9999999999"
}
```

Response:

```
{
  "success": true,
  "data": {
    "otpSent": true
  }
}
```

Verify OTP POST /auth/verify-otp

Request:

```
{
  "mobile": "9999999999",
  "otp": "123456"
}
```

Response:

```
{
  "success": true,
  "data": {
    "customerId": "...",
    "accessToken": "...",
    "refreshToken": "..."
  }
}
```

5.2 Staff Authentication

Staff authentication stays password-based and applies to staff roles only.

Register POST /auth/register

Login POST /auth/login

Refresh POST /auth/refresh

Logout POST /auth/logout

Me **GET** /auth/me

5.3 Password Recovery

Password recovery applies only to:

- Staff
- Kitchen Staff
- Cleaning Staff
- Restaurant Admin
- Super Admin

Customers should **not** have a password reset flow.

Forgot Password **POST** /auth/forgot-password

Request:

```
{
  "email": "staff@restaurant.com"
}
```

Response:

```
{
  "success": true,
  "data": {
    "otpSent": true
  }
}
```

An OTP must be sent to the registered email address of the user.

Verify Reset OTP **POST** /auth/verify-reset-otp

Request:

```
{
  "email": "staff@restaurant.com",
  "otp": "123456"
}
```

Response:

```
{
  "success": true,
  "data": {
    "resetToken": "..."
  }
}
```

Reset Password POST /auth/reset-password

Request:

```
{
  "resetToken": "...",
  "newPassword": "StrongPassword123"
}
```

Response:

```
{
  "success": true
}
```

Password Reset Flow

1. User enters registered email address.
2. System sends OTP to that email address.
3. User verifies the OTP.
4. System issues a temporary reset token.
5. User submits a new password using the reset token.
6. Password is securely updated.

Security Exceptions:

- **429 RATE_LIMIT_EXCEEDED:** Thrown on OTP cooldown violation, daily/hourly request cap limit violation, or failure attempts limit violation.
- **403 FORBIDDEN:** Thrown if trying to request or verify an OTP when a mobile number is in a 15-minute lockout block.

Typical register body

```
{
  "name": "Om",
  "email": "om@example.com",
  "mobile": "9999999999",
  "password": "StrongPassword123"
}
```

6. Public APIs

Method	Path	Purpose
GET	/public/restaurants/:slug	Public restaurant details
GET	/public/menu/:restaurantId	Public menu for a restaurant

Method	Path	Purpose
POST	/public/table-session/validate	Validate QR table session
POST	/public/table-session/create	Create table session
GET	/public/reservations/availability	Check reservation availability
POST	/public/queue/join	Join public queue

Public menu query examples

```
GET /public/menu/:restaurantId?veg=true
GET /public/menu/:restaurantId?category=pizza
GET /public/menu/:restaurantId?spicy=false
GET /public/menu/:restaurantId?search=burger
```

7. Customer APIs

7.1 Customer session

Method	Path	Purpose
GET	/customer/session	Get current table session
PATCH	/customer/session/extend	Extend session
POST	/customer/session/end	End session

7.2 Customer menu

Method	Path	Purpose
GET	/customer/menu/categories	Get menu categories
GET	/customer/menu/items	Get menu items
GET	/customer/menu/items/:id	Get single menu item

Menu query examples

```
GET /customer/menu/items?veg=true
GET /customer/menu/items?category=pizza
GET /customer/menu/items?available=true
GET /customer/menu/items?popular=true
GET /customer/menu/items?recommended=true
GET /customer/menu/items?search=pasta
GET /customer/menu/items?priceMin=100&priceMax=500
GET /customer/menu/items?sortBy=price
```

7.3 Customer cart

Method	Path	Purpose
GET	/customer/cart	Get cart
POST	/customer/cart/items	Add item to cart
PATCH	/customer/cart/items/:itemId	Update cart item
DELETE	/customer/cart/items/:itemId	Remove cart item
DELETE	/customer/cart	Clear cart

7.4 Customer orders

Method	Path	Purpose
POST	/customer/orders	Place order
GET	/customer/orders	List orders
GET	/customer/orders/:id	Get single order
POST	/customer/orders/:id/reorder	Reorder
POST	/customer/orders/:id/cancel	Cancel order

Order query examples

GET /customer/orders?status=PREPARING

GET /customer/orders?page=1&limit=10

7.5 Customer assistance

Method	Path	Purpose
POST	/customer/requests/waiter	Call waiter
POST	/customer/requests/water	Request water
POST	/customer/requests/cutlery	Request cutlery
POST	/customer/requests/cleaning	Request cleaning
POST	/customer/requests/help	Request assistance

7.6 Customer billing

Method	Path	Purpose
GET	/customer/bill	Get live bill
POST	/customer/bill/request	Request final bill
POST	/customer/bill/coupon	Apply coupon
DELETE	/customer/bill/coupon/:couponId	Remove coupon

7.7 Customer payments

Method	Path	Purpose
POST	/customer/payments/create	Create payment
POST	/customer/payments/verify	Verify payment
GET	/customer/payments/:paymentId/status	Payment status

7.8 Customer feedback

Method	Path	Purpose
POST	/customer/feedback	Submit feedback
GET	/customer/feedback	Feedback history

7.9 Customer loyalty

Method	Path	Purpose
GET	/customer/loyalty	Loyalty wallet
GET	/customer/offers	Available offers
GET	/customer/offers/eligibility	Offer eligibility

8. Service Staff APIs

Method	Path	Purpose
GET	/staff/tables	Table dashboard
GET	/staff/tables/:id	Table details
PATCH	/staff/tables/:id/assign	Assign table
PATCH	/staff/tables/:id/reserve	Mark reserved
PATCH	/staff/tables/:id/occupy	Mark occupied
GET	/staff/queue	Queue dashboard
GET	/staff/queue/:id	Queue details
PATCH	/staff/queue/:id/priority	Update queue priority
GET	/staff/reservations	Reservation list
GET	/staff/reservations/:id	Reservation details
PATCH	/staff/reservations/:id/check-in	Check in reservation
GET	/staff/orders/ready	Food pickup queue
PATCH	/staff/orders/:id/pick	Pick order
PATCH	/staff/orders/:id/serve	Mark served
GET	/staff/requests	Staff requests

Method	Path	Purpose
PATCH	/staff/requests/:id/accept	Accept request
PATCH	/staff/requests/:id/complete	Complete request
POST	/staff/issues/escalate	Escalate issue

Staff table query examples

GET /staff/tables?status=AVAILABLE

GET /staff/tables?floor=1

GET /staff/tables?section=VIP

9. Kitchen APIs

Kitchen dashboard and order flow

Method	Path	Purpose
GET	/kitchen/dashboard	Kitchen dashboard
GET	/kitchen/orders	Kitchen orders list
GET	/kitchen/orders/:id	Kitchen order details
PATCH	/kitchen/orders/:id/accept	Accept order
PATCH	/kitchen/orders/:id/start	Start cooking
PATCH	/kitchen/orders/:id/ready	Mark ready
PATCH	/kitchen/orders/:id/delay	Delay order
PATCH	/kitchen/orders/:id/reject	Reject order

Kitchen query examples

GET /kitchen/orders?status=PREPARING

GET /kitchen/orders?priority=HIGH

GET /kitchen/orders?table=12

GET /kitchen/orders?batch=true

Kitchen batching and metrics

Method	Path	Purpose
GET	/kitchen/batches	List batches
GET	/kitchen/batches/:id	Batch details
POST	/kitchen/batches	Create batch
PATCH	/kitchen/batches/:id	Update batch
GET	/kitchen/load	Kitchen load metrics
GET	/kitchen/performance	Chef performance metrics

10. Cleaning APIs

Method	Path	Purpose
GET	/cleaning/tasks	Cleaning task list
GET	/cleaning/tasks/:id	Cleaning task details
PATCH	/cleaning/tasks/:id/start	Start cleaning
PATCH	/cleaning/tasks/:id/complete	Complete cleaning
PATCH	/cleaning/tasks/:id/verify	Verify cleaning

Cleaning query examples

GET /cleaning/tasks?status=PENDING

GET /cleaning/tasks?priority=HIGH

11. Restaurant Admin APIs

11.1 Restaurant settings

Method	Path	Purpose
GET	/admin/restaurant/overview	Restaurant overview
GET	/admin/restaurant/settings	Restaurant settings
PATCH	/admin/restaurant/settings	Update settings

11.2 Tables

Method	Path	Purpose
POST	/admin/tables	Create table
GET	/admin/tables	List tables
GET	/admin/tables/:id	Table details
PATCH	/admin/tables/:id	Update table
DELETE	/admin/tables/:id	Delete table
POST	/admin/tables/bulk	Bulk create tables
POST	/admin/tables/:id/qr	Generate table QR
GET	/admin/tables/:id/qr	Download table QR

11.3 Menu

Method	Path	Purpose
POST	/admin/menu/categories	Create category

Method	Path	Purpose
GET	/admin/menu/categories	List categories
POST	/admin/menu/items	Create menu item
GET	/admin/menu/items	List menu items
GET	/admin/menu/items/:id	Menu item details
PATCH	/admin/menu/items/:id	Update menu item
DELETE	/admin/menu/items/:id	Delete menu item
PATCH	/admin/menu/items/:id/availability	Toggle availability
POST	/admin/menu/items/:id/image	Upload menu image

Admin menu query examples

GET /admin/menu/items?available=true
 GET /admin/menu/items?category=starter
 GET /admin/menu/items?search=burger

11.4 Staff management

Method	Path	Purpose
POST	/admin/staff	Create staff
GET	/admin/staff	List staff
GET	/admin/staff/:id	Staff details
PATCH	/admin/staff/:id	Update staff
DELETE	/admin/staff/:id	Delete staff
POST	/admin/staff/shifts	Shift assignment
GET	/admin/staff/attendance	Attendance list
GET	/admin/staff/performance	Performance metrics

11.5 Offers and loyalty

Method	Path	Purpose
POST	/admin/offers	Create offer
GET	/admin/offers	List offers
GET	/admin/offers/:id	Offer details
PATCH	/admin/offers/:id	Update offer
DELETE	/admin/offers/:id	Delete offer
GET	/admin/loyalty/rules	Loyalty rules
POST	/admin/loyalty/rules	Create loyalty rule

11.6 Billing reports

Method	Path	Purpose
GET	/admin/billing/revenue	Daily revenue
GET	/admin/billing/tax	Tax reports
GET	/admin/billing/orders	Order reports
GET	/admin/billing/discounts	Discount reports
GET	/admin/billing/payments	Payment reports

11.7 Analytics

Method	Path	Purpose
GET	/admin/analytics/revenue	Revenue analytics
GET	/admin/analytics/peak-hours	Peak hours
GET	/admin/analytics/repeat-customers	Repeat customers
GET	/admin/analytics/kitchen	Kitchen analytics
GET	/admin/analytics/table-utilization	Table utilization
GET	/admin/analytics/customer-retention	Customer retention

Revenue analytics query examples

GET /admin/analytics/revenue?from=2026-01-01&to=2026-01-31
 GET /admin/analytics/revenue?groupBy=day
 GET /admin/analytics/revenue?groupBy=month

11.8 Inventory

Method	Path	Purpose
GET	/admin/inventory	Ingredient list
POST	/admin/inventory	Add ingredient
PATCH	/admin/inventory/:id	Update ingredient
GET	/admin/inventory/alerts	Stock alerts

12. Super Admin APIs

12.1 Platform and tenant management

Method	Path	Purpose
GET	/super-admin/platform/overview	Platform overview
GET	/super-admin/restaurants	Restaurant list
GET	/super-admin/restaurants/:id	Restaurant details

Method	Path	Purpose
PATCH	/super-admin/restaurants/:id/approve	Approve restaurant
PATCH	/super-admin/restaurants/:id/suspend	Suspend restaurant
DELETE	/super-admin/restaurants/:id	Delete restaurant

Restaurant list query examples

GET /super-admin/restaurants?status=ACTIVE

GET /super-admin/restaurants?plan=PRO

GET /super-admin/restaurants?search=pizza

12.2 Plans, analytics, and platform controls

Method	Path	Purpose
POST	/super-admin/plans	Create subscription plan
GET	/super-admin/plans	List plans
PATCH	/super-admin/plans/:id	Update plan
GET	/super-admin/analytics/revenue	Platform revenue analytics
GET	/super-admin/analytics/tenants	Active tenant analytics
GET	/super-admin/system/monitoring	System monitoring
GET	/super-admin/audit-logs	Audit logs
GET	/super-admin/feature-flags	Feature flags
PATCH	/super-admin/feature-flags/:id	Update feature flag

Audit log query examples

GET /super-admin/audit-logs?actorId=123

GET /super-admin/audit-logs?action=DELETE

GET /super-admin/audit-logs?from=2026-01-01&to=2026-01-10

13. Shared Utility APIs

Method	Path	Purpose
GET	/notifications	List notifications
PATCH	/notifications/:id/read	Mark notification read
PATCH	/notifications/read-all	Mark all notifications read
POST	/uploads	File upload
GET	/search?q=pizza	Global search

14. Webhook APIs

Webhook routes are reserved for future third-party integrations such as:

- payment gateway callbacks
- SMS callbacks
- email delivery callbacks
- external automation callbacks

If implemented later, keep webhook verification strict and signature-based.

15. Socket Testing APIs

Socket testing is used to verify live events for:

- order updates
- kitchen queue updates
- billing updates
- reservation updates
- notifications
- table state updates
- staff request updates

Recommended event groups:

- order:new
 - order:status-updated
 - kitchen:batch-created
 - kitchen:batch-updated
 - table:status-updated
 - notification:new
 - session:created
 - session:expired
-

16. Health and Monitoring APIs

Method	Path	Purpose
GET	<code>/health</code>	Liveness check
GET	<code>/ready</code>	Readiness check
GET	<code>/version</code>	Version check

17. QR Session Security Notes

- QR creates a temporary table session
- Session expires after dining/payment
- Old URL cannot reopen the table
- Reusing an expired session must return a clear error

Example:

```
{  
  "success": false,  
  "error": {  
    "code": "TABLE_SESSION_EXPIRED",  
    "message": "Please scan another QR code."  
  }  
}
```

18. Postman Environment Variables

Use these in the workspace collection:

```
baseUrl  
accessToken  
refreshToken  
restaurantId  
tableId  
sessionId  
orderId  
menuItemId  
reservationId  
staffId  
customerId  
paymentId  
billId  
offerId  
batchId  
notificationId
```

19. Recommended Request Flow for Testing

1. Auth
2. Public QR session create/validate
3. Customer menu browse
4. Cart add/update/remove
5. Order place and order status tracking

6. Kitchen accept/start/ready
 7. Staff pick/serve
 8. Billing request and coupon flow
 9. Payment create/verify
 10. Session close
 11. Cleaning task verification
 12. Admin analytics and inventory checks
 13. Super admin tenant checks
 14. Notification and socket event checks
-

20. Workspace and Run Notes

This project uses an npm workspace setup. Recommended commands:

```
npm install
npm run dev --workspace backend
npm run dev --workspace frontend
npm run typecheck
npm run verify:phase1 --workspace backend
docker-compose up --build
```

21. Summary

This API documentation must stay aligned with:

- backend route ownership
- PRD expectations
- session-first customer UX
- JWT-based staff/admin auth
- role-wise Postman collection structure

If an endpoint is changed, the documentation and the Postman collection must be updated at the same time.